```
int& getValue() {
   static int a = 10;
   return a;
}
int main() {
   getValue() = 22;    // modify through function
   cout << getValue();   // 22
}
```

```
const int& getValue() {
   static int a = 10;
   return a;
}
int main() {
   // getValue() = 22;   read-only
   cout << getValue();   // 10
}
```

```
int a = 10;
int& getValue() {
   return a;
}
int main() {
   getValue() = 50;    // modify through function
   cout << a;   // 50
}
```

```
int a = 10;
const int& getValue() {
   return a;
}
int main() {
   // getValue() = 50;   read-only
   cout << getValue();   // 10
}
```

```
int& getValue(int& x) {
   return x;
}
int main() {
   int a = 10;
   getValue(a) = 99;    // modify through function
   // cout << getValue(20);   // 20
   cout << a;   // 99
}
```

```
const int& getValue(const int& x) {
   return x;
}
int main() {
   int a = 10;
   // getValue(a) = 99;   read-only
   cout << getValue(a);    // 10
   cout << getValue(20);   // 20
}
```

```
const int& func() {
   const int x = 10;    // local variable
   return x;    // ❌ DANGEROUS
}
const int& r = func();   // ❌ r is dangling
```

```
const int& func() {
   return 20;    // temporary
}

const int& r = func();   // ❌ still unsafe!
```

const int& z = 20; // ✅ valid A const reference can bind to both lvalues and rvalues

- 👉 const prevents modification
- 👉 which makes binding to a temporary safe
- 👉 and then C++ allows lifetime extension

```
const int w = 10;   // ✅ valid
const int& x = w;
int& y = 20;   // ❌ not allowed
```

int a=10; int &b=a; cout<<a<<b; /* b=20; */ **a=20; cout<<a<<b;** // 10102020